Yongjae Kwon 권용재

Frontend Developer

EMAIL **yongjae116@gmail.com** | PHONE **+82 10-9470-1704** | LOCATION **Yongin, South Korea**
GITHUB **github.com/YongjaeKwon** | PORTFOLIO **www.yongjaekwon.com**

SUMMARY

A frontend developer who keeps users on track through complex business work — surfacing input, progress, and error states clearly, and wiring up the APIs and data handling each screen needs. On production Vue/JavaScript systems I have owned everything from the UI to Spring APIs, SQL, and deployment; in React/TypeScript projects I built dynamic forms, REST-backed dashboards, and user-flow tests. I pin down task order and edge cases with operations staff, and keep fixing what real usage reveals.

SKILLS

Frontend	WORK	JavaScript, Vue, HTML, CSS, WebSquare5
	PROJECTS	TypeScript, React, React Router, Next.js, Vite, Tailwind CSS
		In ReachRich I connected FastAPI responses to TypeScript types and shared query helpers, splitting the dashboard, trading journal, stock ratings, and weekly reports into React Router routes.
State · Forms		React Hook Form, TanStack Query, Redux Toolkit, Pinia
		In SSAFAST, React Hook Form's FormProvider and useFieldArray managed repeated and nested inputs; TanStack Query handled server data and post-save refreshes; Redux Toolkit held shared UI state like theme and toasts.
Responsive UI		CSS Grid, Flexbox, Tailwind CSS
		My portfolio keeps reading order on small screens — a single column for cards, details, and inputs — expanding to two columns only on wide screens.
Server · DB	SERVER	Java, Spring Boot, Spring MVC, Python, FastAPI, MyBatis
	DB	MariaDB, MySQL, Oracle, SQLite
		At work I build Spring APIs and MyBatis SQL together with each screen's permission, state, and save conditions; in ReachRich I connected FastAPI query APIs and SQLite data to React screens.
Test · Delivery		Vitest, Testing Library, pytest, Git, GitHub Actions, Jenkins, SVN
		With Vitest and Testing Library I verified loading and empty states, refetch on period change, polling that pauses in hidden tabs and resumes on return, and duplicate-submit prevention. GitHub Actions runs the frontend tests and TypeScript build.

EXPERIENCE

TG Narae LLC — Web Developer · Solutions Team

Jun 2024 – Present · Full-time

On a B2B partner portal and the Seoul Metropolitan Office of Education's device-management system, I shape requirements with operations staff and own UI, REST API, and SQL development through user acceptance, deployment, and operations.

B2B Partner Portal (PPS)

Feb 2025 – Present

A portal where headquarters and about 500 partner companies handle partner registration and contracts, plus training, certification, and documentation for 824 field engineers. I own the Vue screens, Spring APIs and SQL, external-system integrations, and operations.

Tech | Vue · JavaScript · Java 21 · Spring Boot · MyBatis · MariaDB · Oracle

Highlights

Stopping attachment cross-linking caused by leaked shared Vue state

Problem	When registering a new field engineer, the previously viewed engineer's attachment group lingered on screen, so the previous person's files could be linked to the new record.
Approach	Rather than just clearing values on open, give every Vue instance fresh state — and have the server block wrong links again at save time.
Implementation	Replaced shallow copies of a shared object with a state-factory function, resetting attachment state on register, detail view, and failed lookups. The server now rejects saves referencing an attachment group owned by another engineer.
Result	Removed the root cause of stale attachments reappearing after navigation, and made sure a UI slip can no longer corrupt stored links.

Async processing and progress UI for bulk downloads

Problem Collecting and zipping attachments for 300–400 engineers in one request kept the screen waiting for minutes, with no way to tell progress from failure.

Approach Instead of only speeding up compression, separate the HTTP request from the long-running file job and let the screen follow the job's status.

Implementation A Spring async job returns a job ID immediately; the Vue screen polls waiting / running / done / failed status. The job ID is kept in the browser so the same job can be found again after a refresh.

Result Users can track progress and download only completed files, without the browser holding an HTTP connection open.

Education Device Management System (TSMS)

Sep 2025 – Present

Manages about 110,000 devices for the Seoul Metropolitan Office of Education's one-device-per-student program — from registration through delivery, installation, repairs, and inspections. I own the WebSquare screens, Spring APIs and SQL, external integrations, user acceptance, and deployment.

Tech | WebSquare5 · JavaScript · Java 11 · Spring MVC · MyBatis · MariaDB

Highlights

Building the school inspection flow from scheduling to sign-off

Problem Schedule approval, roster checks, checklists, e-signatures, and confirmations had to flow together — and multiple staff on the same school risked duplicate approvals or overwriting earlier results.

Approach Connect the screens in the order staff actually work, but have the server and DB re-verify current state at save time.

Implementation Linked the WebSquare screens along the real task order and locked school data before approval. Results save only if the version number read at load time still matches; re-inspections preserve earlier confirmations and device records.

Result One continuous flow from scheduling to sign-off, with duplicate processing and stale-screen overwrites blocked on the server.

Public enrollment screens for parents across four education offices

Problem Parents handle rental consent, delivery-date booking, and QR handout checks on public screens with no login — so late submissions, duplicates, and mobile rendering issues all had to be handled in the UI.

Approach Unlike admin screens, put guidance text, error screens, and deadline locking first, and design mobile-first by default.

Implementation Built consent, QR check, and delivery-booking screens for four education offices (Jeju, Sejong, Gyeonggi, Gangwon), handling submission locks after deadlines, back-navigation guards, a dedicated error screen on parse failures, and QR codes unreadable in dark mode.

Result Parents complete enrollment from the guidance alone; schedule changes and deadline requests during live periods were applied same-day.

SELECTED PROJECTS

ReachRich · React operations dashboard

Mar 2026 – Present · Personal

Connected FastAPI query APIs to a React + TypeScript UI — assets, trading journal, stock ratings, and weekly reports as routes. Polling pauses in hidden tabs and refreshes on return; key data states and query flows are verified with Vitest and Testing Library.

Tech | React · TypeScript · Vite · Vitest · Testing Library · FastAPI · GitHub Actions

SSAFAST · API spec & testing collaboration tool

Apr 2023 – May 2023 · Team of 6

As the frontend / UI-UX developer, I built the flow from API spec authoring to request execution and result review — React Hook Form for repeated and nested inputs, TanStack Query for server data, Redux Toolkit for shared UI state.

Tech | React · TypeScript · Next.js · React Hook Form · TanStack Query · Redux Toolkit

Personal portfolio · responsive project archive

Mar 2025 – Present · Personal

Built from scratch with Vue 3, TypeScript, Vite, and Tailwind CSS. Single-column on small screens, two-column on wide; the shared detail modal supports keyboard navigation, focus restoration, and reduced motion.

Tech | Vue 3 · TypeScript · Vite · Tailwind CSS · Vitest

EDUCATION

Training	**Samsung SW Academy For Youth (SSAFY), 8th cohort** · **Web development program**	Jul 2022 – Jun 2023
Education	**Ajou University** · **B.S. in e-Business (transfer)**	Mar 2018 – Aug 2020
Education	**California State University, Chico** · **Business Administration, attended before transfer**	Jan 2014 – May 2015
Certification	**SQLD (SQL Developer)** · **Korea Data Agency**	Sep 2024